Sentence Splitter

Fine-tuning XLM-RoBERTa-large for Sentence Boundary Detection

Luca Tam
2045168
Sapienza University of Rome
Multilingual Natural Language Processing

1 Introduction

This report describes a sentence boundary detection system built by fine-tuning **XLM-RoBERTa-large** (560M parameters) as a per-token binary classifier. The model is trained on 8 Universal Dependencies treebanks (4 English, 4 Italian) and achieves a macro F1 of **0.9863**, improving over the NLTK Punkt baseline (0.9411) by +4.5% and spaCy (0.9519) by +3.4% absolute.

The full source code, training scripts, and evaluation pipeline are available at:

https://github.com/LucaTamSapienza/sentence_splitter

2 Project Structure

```
sentence_splitter/
  src/                    # Python package
    __init__.py           #   Package manifest + public API
    # --- Core library ---
    model.py              #   XLM-R classifier + focal loss
    data.py               #   Data parsing, windowing, datasets
    train.py              #   Training loop (fp16, early stopping)
    inference.py          #   Sliding-window prediction pipeline
    evaluate.py           #   Metrics: boundary F1, exact match
    ensemble.py           #   Multi-model blending + optimization
    rules.py              #   Rule-based heuristics (abbreviations)
    utils.py              #   Config, seeding, logging helpers
    # --- CLI scripts ---
    train_xlmr.py         #   Train on UD data
    evaluate_xlmr.py      #   Threshold tuning + test evaluation
    predict.py            #   Inference on files or folders
    run_baselines.py      #   NLTK + spaCy baselines
    optimize.py           #   Ensemble weight grid search
    eval_test.py          #   Evaluate on JSONL gold test sets
    build_test.py         #   Convert <EOS> files to JSONL
  configs/xlmr.yaml       # Hyperparameter configuration
  input/                  # Place .txt files here for inference
  output/                 # Split results appear here
  checkpoints/            # Model checkpoint (via download_model.py)
  UD_*/                   # 8 UD treebanks (via --data flag)
  data/                   # JSONL gold test sets (via --data flag)
  download_model.py       # Download model + dataset from HuggingFace
  requirements.txt
```

3 Method

3.1 Architecture

The model applies a linear classification head on top of each token representation produced by XLM-RoBERTa-large:

$$\hat{y}_i = \sigma\big(\mathbf{w}^\top \operatorname{Dropout}(\mathbf{h}_i) + b\big), \qquad \mathbf{h}_i \in R^{1024}$$

where $\mathbf{h}_i$ is the hidden state of the i-th subtoken, σ is the sigmoid function, and $\hat{y}_i$ is the probability that token i ends a sentence.

3.2 Sliding Windows

Documents are split into overlapping windows of 510 content tokens (plus `[CLS]` / `[SEP]`) with a stride of 256 tokens (50% overlap). Predictions in overlapping regions are averaged, providing ensemble-like robustness without multiple models.

3.3 Training Details

- **Loss**: Focal binary cross-entropy (α=0.75, γ=2.0) to handle the ∼1:25 boundary-to-non-boundary class imbalance.
- **Optimizer**: AdamW, learning rate 2×10^{-5}, weight decay 0.01, linear warmup (10%) followed by linear decay.
- **Batch size**: 16 with gradient accumulation of 2 (effective 32).
- **Precision**: FP16 mixed precision.
- **Early stopping**: patience of 3 epochs on dev boundary F1.
- **Sampling**: Temperature-weighted (T=2.0) across all 7 training treebanks to prevent large corpora from dominating.
- **Seed**: 42 for full reproducibility.

4 Reproducibility Guide

4.1 Environment Setup

```
git clone https://github.com/LucaTamSapienza/sentence_splitter.git
cd sentence_splitter
pip install -r requirements.txt
python download_model.py
```

The last command downloads the fine-tuned checkpoint (∼2.2 GB) from HuggingFace (`Famezz/xlmr-sentence-splitter`) into `checkpoints/best_xlmr_model.pt`.

For GPU support with CUDA 12.1:

```
pip install torch --index-url https://download.pytorch.org/whl/cu121
```

To run the NLTK and spaCy baselines, also execute:

```
python -m nltk.downloader punkt_tab
python -m spacy download en_core_web_sm
python -m spacy download it_core_news_sm
```

4.2 Training

The training data consists of `.sent_split` files from 8 Universal Dependencies treebanks, where sentence boundaries are marked with `<EOS>` tokens. Download the dataset first, then train:

```
python download_model.py --data
```

```
python src/train_xlmr.py \
    --data_dir ./ \
    --output_dir checkpoints \
    --model_name_or_path xlm-roberta-large
```

The best checkpoint is saved to `checkpoints/best_xlmr_model.pt`. Training takes approximately 1–2 hours on a single GPU (e.g., NVIDIA A100).

4.3 Evaluation and Threshold Tuning

After training, evaluate on the dev/test splits and sweep thresholds:

```
python src/evaluate_xlmr.py \
    --data_dir ./ \
    --model_path checkpoints/best_xlmr_model.pt \
    --model_name_or_path xlm-roberta-large \
    --predictions_dir outputs/predictions
```

The script sweeps thresholds on dev and selects the best one (typically t=0.65), then evaluates on all test sets.

4.4 Inference on New Text

Place one or more `.txt` files in the `input/` folder and run:

```
python src/predict.py \
    --input input/ \
    --output output/ \
    --model_path checkpoints/best_xlmr_model.pt
```

Each file in `input/` is processed and the split result is written to `output/` with the same filename. A single file can also be passed directly (`-input file.txt -output result.txt`).

For example, given `input/example.txt`:

```
Dr. Smith met with prof. A. Rossi at 10:30 a.m. in
Washington D.C. to discuss the new agreement. The meeting
lasted two hours.
```

The script produces `output/example.txt`:

```
1) Dr. Smith met with prof. A. Rossi at 10:30 a.m. in
   Washington D.C. to discuss the new agreement.

2) The meeting lasted two hours.
```

4.5 Running Baselines

To reproduce the NLTK and spaCy baselines for comparison:

```
python src/run_baselines.py --split test --verbose
```

5 Results

Table 1 reports boundary-level F1 scores on all 8 test sets.

The fine-tuned XLM-RoBERTa model outperforms both baselines by a significant margin across nearly all datasets, with the largest gains on harder corpora such as EN-EWT (+15.4% over NLTK) and IT-VIT (+5.5% over spaCy). The model handles both English and Italian with a single multilingual checkpoint.

Dataset	NLTK	spaCy	XLM-R (t=0.65)
EN-EWT	0.803	0.819	0.957
EN-GUM	0.919	0.923	0.979
EN-ParTUT	0.993	0.987	0.997
EN-PUD	0.991	0.997	0.988
IT-ISDT	0.954	0.991	0.997
IT-MarkIT	0.917	0.984	0.997
IT-ParTUT	0.993	0.993	1.000
IT-VIT	0.958	0.921	0.976
Macro F1	**0.9411**	**0.9519**	**0.9863**

Table 1: Boundary-level F1 on all 8 UD test sets. Threshold tuned on dev (t=0.65).

5.1 Real-World Gold Test Sets

Beyond the UD benchmarks, we manually created gold-annotated test sets to evaluate the model on realistic, challenging text that standard treebanks do not cover. These are available in the `data/` directory as JSONL files built with `src/build_test.py`. They can be evaluated with:

```
python src/eval_test.py \
    --input data/full_test.jsonl \
    --checkpoint checkpoints/best_xlmr_model.pt \
    --threshold 0.65
python src/eval_test.py \
    --input data/Gold.jsonl \
    --checkpoint checkpoints/best_xlmr_model.pt \
    --threshold 0.65
```

The test sets include:

- **hard_test** (20 sentences): Edge cases such as abbreviations (Dr., Ph.D., U.S., S.p.A., art.), ellipses, multi-line sentences, numbered lists, and headers without punctuation.
- **extended_test** (62 sentences): Academic and technical language, URLs, mathematical notation ($O(n \log n)$, $f(x)$), Italian legal references (d.lgs., Corte di Cassazione), citations (Smith et al., 2023), and informal conversational text.
- **Gold** (165 sentences): A comprehensive real-world test set covering Italian legal prose (sentenze, ordinanze, T.A.R., Codice Privacy), medical/scientific text (E. coli, temperature conversions), financial language (fatturato, EUR/USD), system logs, regex patterns, bibliographic citations, nested parenthetical structures, and mixed Italian/English content.

Test set	Sents	P	R	F1	ExMatch
hard_test	20	1.000	0.950	0.974	0.950
extended_test	62	0.938	0.984	0.961	0.894
Gold	165	0.896	0.988	0.939	0.803

Table 2: Results on manually annotated gold test sets (t=0.65). ExMatch = fraction of sentences exactly matching the gold segmentation.

The model maintains strong performance on these out-of-domain texts. Most errors involve over-splitting at ambiguous punctuation in complex legal abbreviations or enumerated lists cases where even human annotators may disagree. The high recall ($\geq$0.95) shows that the model rarely misses a true sentence boundary.

6 Conclusion

Fine-tuning XLM-RoBERTa-large with focal loss, sliding-window inference, and temperature-weighted multilingual sampling yields a sentence splitter that substantially outperforms NLTK Punkt and spaCy, achieving 0.9863 macro F1 on 8 Universal Dependencies test sets. The full pipeline: training, evaluation, and inference is reproducible from the public repository.